Whitebox Vulnerability Report

Target Application: Leantime (Local Instance)

Scope: Authentication Tokens, Profile Handling, File Processing, and Access Control Mechanisms

1. Executive Summary

A security assessment was performed on the local Leantime installation to evaluate user profile handling, session management mechanics, cryptographic token generation, and asset retrieval controls. The assessment identified several systemic architectural weaknesses, including deterministic account recovery tokens, unvalidated avatar file processing, insecure direct object references in media retrieval, and non-cryptographic pseudo-random number generation for form tokens. This report outlines the technical findings, underlying technologies, step-by-step verification proofs, and remediation guidance.

2. Technology Stack & Architectural Overview

The Leantime application relies on a hybrid architecture blending native PHP design patterns with modern components from the Laravel and Symfony frameworks:

- **Core Routing & Dispatch:** Operates a dual routing mechanism. Legacy convention routes are handled via `Leantime\Core\Controller\Frontcontroller`, while native Laravel routes are managed through the framework's routing layer and security middleware (e.g., `CheckPermissions`).
- **HTTP & Request Management:** Uses `Symfony\Component\HttpFoundation` to manage incoming HTTP requests, payloads, redirection parameters, and response headers.
- **Database Management & Query Builder:** Implements the Illuminate Database component (`ConnectionInterface`) as an abstraction layer over a relational MySQL database containing tables such as `zp_user` and `zp_clients`.
- **Authorization Engine:** Enforces access control through attribute-based annotations (`#[RequiresPermission]`), evaluated globally or entity-scoped via `PermissionEnforcer` and `PermissionService`. Roles and numerical mappings (e.g., Read Only `5`, Admin `40`, Owner `50`) are centralized within `Leantime\Domain\Auth\Models\Roles`.

3. **Detailed Vulnerability Findings & POC

Finding 1: Insecure Deterministic Token Generation in `getUserBySha`

Overview & Impact

- **Severity:** Medium
- **CWE Mapping:** CWE-330: Use of Insufficiently Random Values / CWE-327: Use of a Broken or Risky Cryptographic Algorithm
- **Affected Files:**
  - `app/Domain/Users/Repositories/Users.php`

Technical Description

Account recovery and invitation lookups rely on a custom query method:

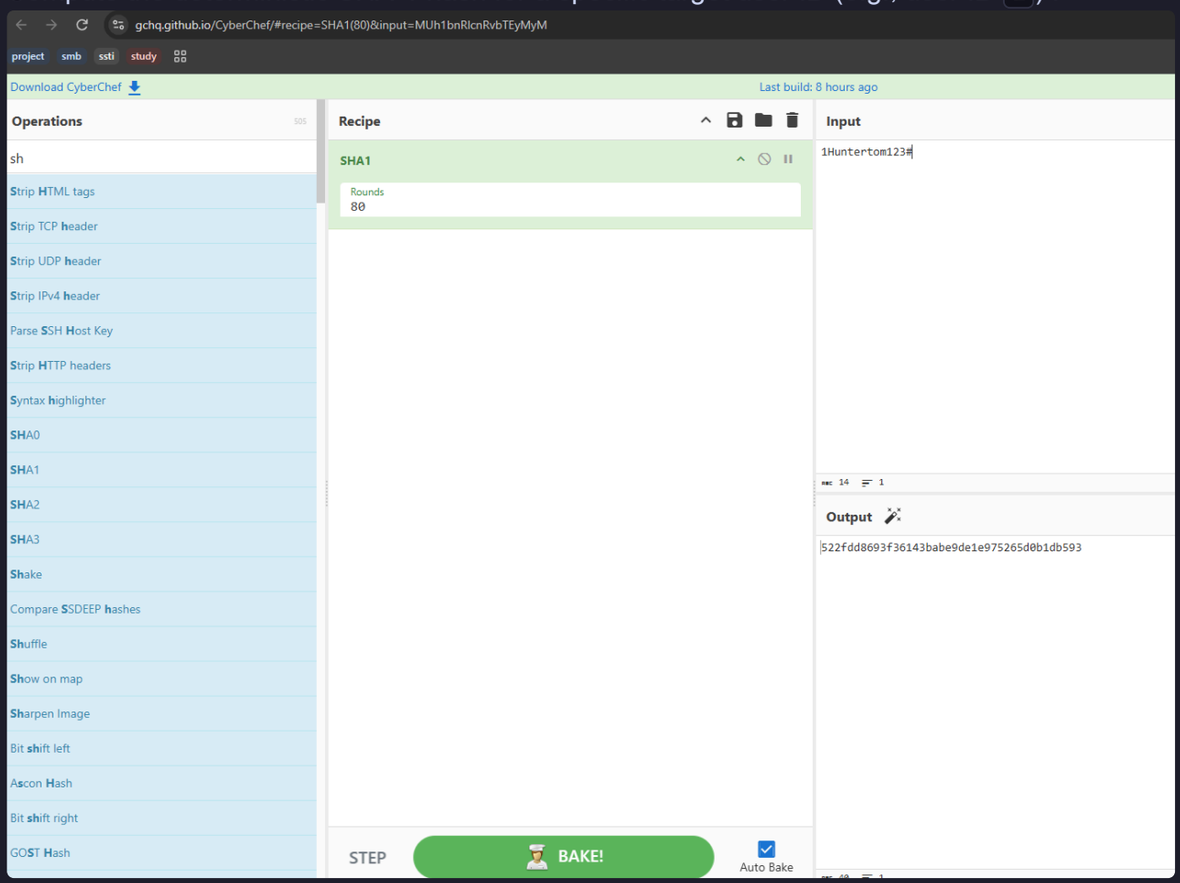
```
public function getUserBySha($hash): array|false
{
    $result = $this->connection->table('zp_user')
        ->whereRaw('SHA1(CONCAT(id, ?)) = ?', [$this->config->sessionPassword,
$hash])
        ->first();
```

```
    return $result ? (array) $result : false;
}
```

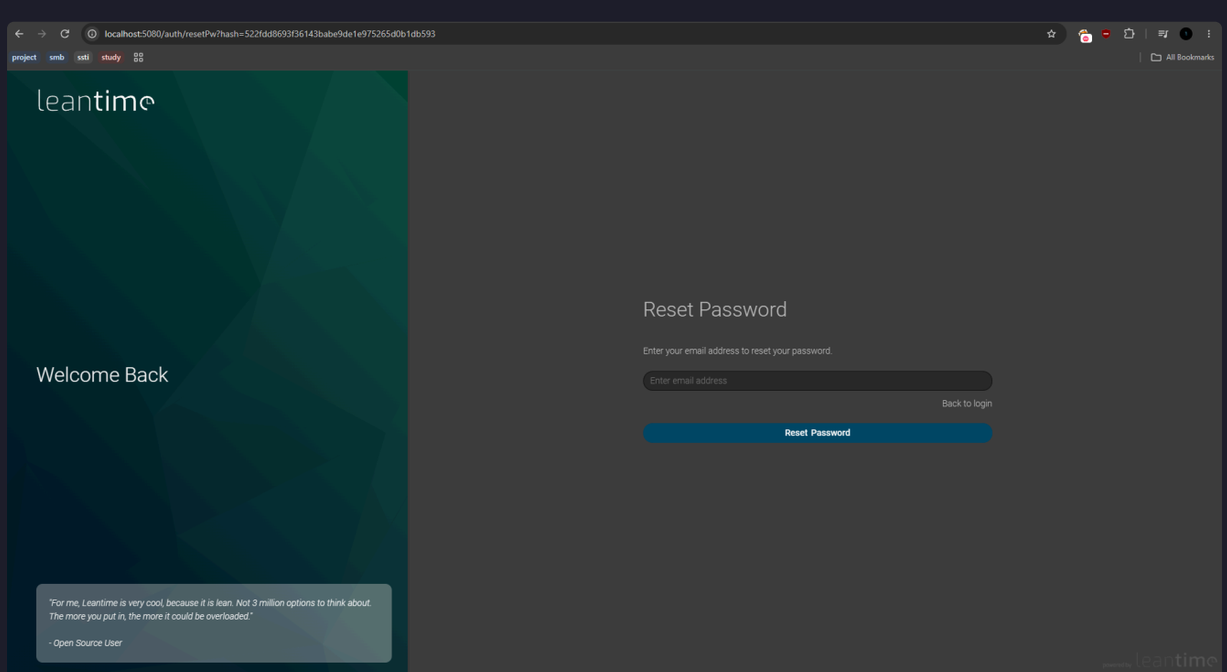
The resulting hash is generated by concatenating an incrementing user ID with a static application secret (`sessionPassword`) and hashing the combination using SHA-1. Because SHA-1 is cryptographically obsolete, fast to compute, and relies on predictable sequential user IDs, knowledge or leakage of the static configuration secret allows an attacker to pre-compute valid reset hashes for any account without rate-limiting or expiration validation.

Step-by-Step Proof of Concept (PoC)

1. Identify the static application configuration string (`sessionPassword`) from environment parameters or setup files.
2. Compute the deterministic SHA-1 hash for a specific target user ID (e.g., user ID `2`) :



3. Append the resulting hash string to the password reset URL endpoint (`/auth/resetPw?hash=<hash>`).
4. Submit the request and observe that the application accepts the deterministic string and initializes a valid recovery session.



Remediation Guidance

Eliminate deterministic token derivation entirely. Implement cryptographically secure, high-entropy random tokens stored as salted hashes inside a dedicated database table with explicit expiration timestamps and single-use flags.

Finding 2: Insecure File Upload Processing in `ProfileImage`

Overview & Impact

- **Severity:** Medium
- **CWE Mapping:** CWE-434: Unrestricted Upload of File with Dangerous Type
- **Affected Files:**
  - `app/Domain/Users/Controllers/ProfileImage.php`

Technical Description

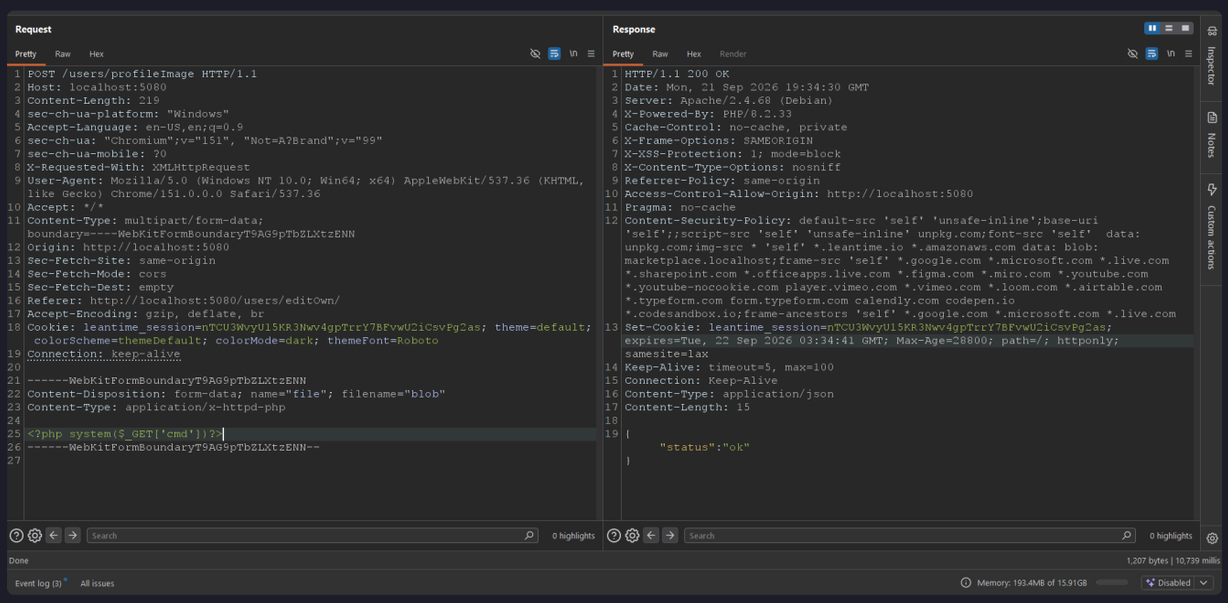
During profile picture processing, the application controller automatically forces a generic extension override on uploaded files:

```
$_FILES['file']['name'] = 'userPicture.png';
$this->userService->setProfilePicture($_FILES, session('userdata.id'));
```

However, the upload pipeline fails to perform rigorous server-side content verification, such as inspecting file magic bytes or passing raw contents through an image processing engine (GD or Imagick) to normalize the file. This creates risks if storage directories are ever misconfigured to permit script execution.

Step-by-Step Proof of Concept (PoC)

1. Authenticate as an active user and open the account appearance settings panel.
2. Intercept the avatar upload request via an HTTP proxy (such as Burp Suite).
3. Replace the file payload with an arbitrary non-image binary structure while maintaining valid multipart boundaries.
4. Forward the request and observe whether the application stores the asset based solely on the enforced filename extension.



Remediation Guidance

Implement strict MIME-type validation via `finfo`, enforce hard maximum file size limits, and normalize uploaded files by re-encoding them through a graphics extension (`imagecreatefromstring` / `imagepng`) before saving them to a secure, non-executable directory outside the public web root.

Finding 3: Insecure Direct Object Reference (IDOR) & User Enumeration in `ProfileImage::show`

Overview & Impact

- **Severity:** Low-Medium
- **CWE Mapping:** CWE-639: Authorization Bypass Through User-Controlled Key / CWE-203: Information Exposure Through Discrepancy
- **Affected Files:**
  - `app/Domain/Users/Controllers/ProfileImage.php`

Technical Description

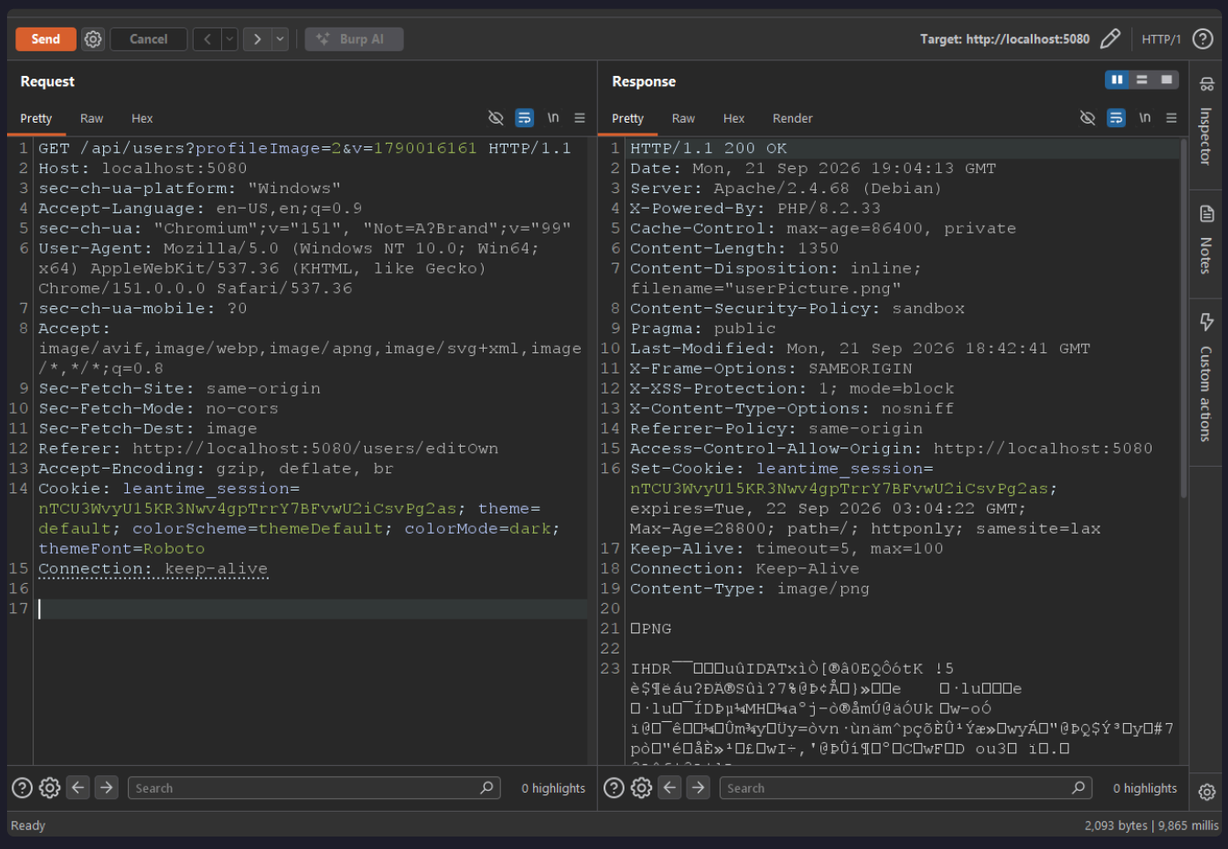
The profile image rendering endpoint accepts an arbitrary user identifier via query strings or dynamic URI segments:

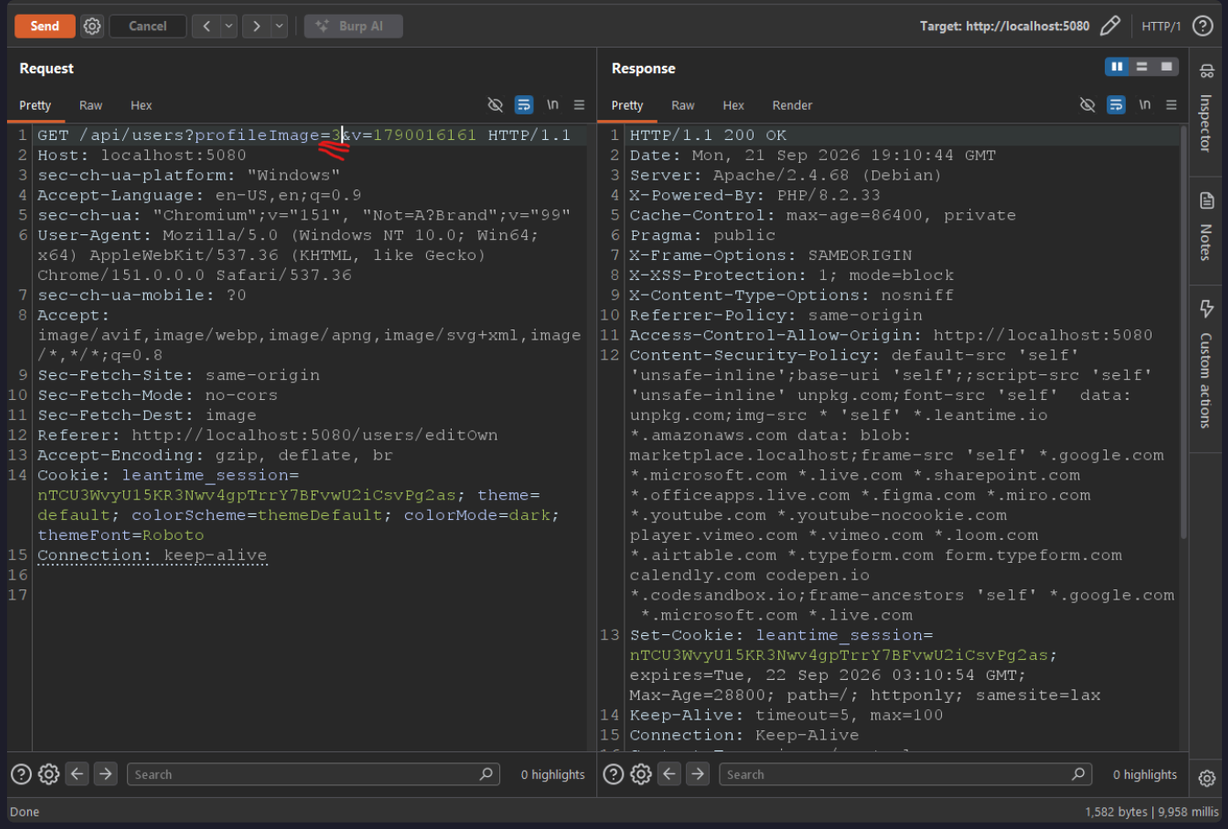
```php
public function show(Request $request, ?string $id = null): ImageResponse
{
    $id = $request->query('profileImage', $id) ?? 'false';
    if ($id === 'me') {
        $id = session('userdata.id');
    }
    return new ImageResponse($this->userService->getProfilePicture($id));
}
```

The controller lacks an access control check verifying whether the calling user possesses authorization to view another user's records. Additionally, differences in HTTP response codes and payload sizes between valid IDs and non-existent IDs facilitate automated user enumeration.

Step-by-Step Proof of Concept (PoC)

1. Log into the application using a baseline user account (`role = 5`).
2. Transmit an HTTP GET request targeting a sequential profile asset ID:
   ```http
   GET /users/profileImage/1 HTTP/1.1
   Host: localhost:5080
   ```
3. Issue a separate request for a non-existent identifier (e.g., `/users/profileImage/999999`).
4. Compare response headers and error messages to confirm differential behavior enabling user roster discovery.





Remediation Guidance

Enforce authorization checks ensuring that the authenticated user either requests their own profile asset or holds the `users.view` permission before retrieving avatar records.

Finding 4: Non-Cryptographic Pseudo-Random Number Generation for CSRF Tokens

Overview & Impact

- **Severity:** Low
- **CWE Mapping:** CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)
- **Affected Files:** `app/Domain/Users/Controllers/EditUser.php`

Technical Description

In `app/Domain/Users/Controllers/EditUser.php`, the application generates custom anti-CSRF token names and values using PHP's native `str_shuffle()` function combined with a predefined set of permitted characters:

```
$permitted_chars = '0123456789abcdefghijklmnopqrstuvwxyz';
session(['formTokenName' => substr(str_shuffle($permitted_chars), 0, 32)]);
session(['formTokenValue' => substr(str_shuffle($permitted_chars), 0, 32)]);
```

The `str_shuffle()` function relies internally on the Mersenne Twister PRNG (`rand()` / `mt_rand()`), which is a non-cryptographic random number generator. Because Mersenne Twister outputs can theoretically be predicted or brute-forced if a sufficient number of consecutive generated states are observed, this implementation fails to meet modern cryptographic standards. While direct exploitation is partially constrained by browser Same-Origin Policy (SOP) boundaries, relying on non-CSPRNG functions weakens defense-in-depth guarantees for session integrity mechanisms.

Step-by-Step Proof of Concept (PoC)

1. Open the project source code repository and navigate to `app/Domain/Users/Controllers/EditUser.php`.
2. Locate the token generation logic block where `str_shuffle()` is invoked to populate session variables for `formTokenName` and `formTokenValue`.

3. Compare this implementation against standard cryptographic requirements (such as `random_bytes()` or `random_int()`), confirming that the entropy pool is derived from a predictable linear congruential or Mersenne algorithm rather than a CSPRNG.

Remediation Guidance

Replace the custom shuffling logic entirely with cryptographically secure random number generators:

```
session(['formTokenName' => 'ft_' . bin2hex(random_bytes(16))]);
session(['formTokenValue' => bin2hex(random_bytes(32))]);
```

4. Summary Matrix

ID	Vulnerability Summary	Affected Component	Severity	Recommended Fix
1	Deterministic SHA-1 Recovery Tokens	`Users::getUserBySha`	Medium	Implement cryptographically random tokens with database expiry.
2	Unvalidated Avatar Upload Processing	`ProfileImage.php`	Medium	Enforce strict MIME checks and server-side image re-encoding.
3	IDOR & Enumeration in Image Retrieval	`ProfileImage::show`	Low-Medium	Add explicit permission gating for roster asset requests.
4	Non-Cryptographic PRNG for Form Tokens	`EditUser.php`	Low	Replace shuffle mechanics with `random_bytes()`.